SOFTWARE ENGINEERING PROJECT REPORT

IDMatch — Interior-Design Matching Platform

Supplementary Report: TDD, DevOps & CI/CD, Ethics, and Data-Driven Development

An online platform matching renovation customers with interior designers in Singapore.
Implementing user stories ID01–ID08 · BCE architecture · React + Node + TypeScript
Repository: github.com/Beh69/idmatch-platform

Module: _____ | Group: _____
Team members: _____

This report documents the software-engineering practices applied to the IDMatch platform: (1) Test-Driven Development, (2) DevOps and CI/CD, (3) Ethical considerations, (4) Data-Driven Software Development, (5) large-scale test-data generation for the live demonstration, (6) the traceability from the BCE sequence diagrams to the implementing code and the website, and (7) a full-program verification confirming the system works. Every section is grounded in the project's actual codebase rather than theory alone; file paths, commands, and test output refer to the working application.

1 Test-Driven Development (TDD)

Test-Driven Development follows a short, repeating **Red** → **Green** → **Refactor** cycle: write a failing test that specifies the desired behaviour (RED), write the minimum code to make it pass (GREEN), then clean up the design (REFACTOR). We applied this to a new, genuinely useful feature for the platform, mirroring the Calculator `multiply()` exercise but on real project code.

1.1 The chosen feature: an interest score

Key aspect 5 of the brief requires the platform to *track interest in an IDP* (how many times it is viewed and shortlisted). We introduced `interestScore(views, shortlists)` that combines these into a single ranking metric. A shortlist is a far stronger signal of buying intent than a casual view, so it is weighted more heavily (view = 1 point, shortlist = 5 points). A companion function `rankByInterest()` orders portfolios by that score. Both live in `src/shared/interestScore.ts` and are unit-tested in `tests/interestScore.test.ts`.

1.2 Step 1 — Define the function with an empty body

Exactly as in the Calculator example, the function is first declared with no real implementation (it returns 0), so the tests compile but nothing works yet:

```
// src/shared/interestScore.ts  (initial RED stub)
export function interestScore(views, shortlists) {
  return 0;        // not implemented yet
}
export function rankByInterest(items) {
  return items;    // not implemented yet
}
```

1.3 Step 2 — Write the test cases first (each in its own function)

Eight independent test cases specify the behaviour before any code exists — zero activity, views only, shortlists only, the additive combination, the shortlist-outweighs-view rule, rejection of negative inputs, correct ranking order, and immutability of the input array:

```
// tests/interestScore.test.ts  (excerpt)
test("score of zero activity is zero", () => {
  assert.equal(interestScore(0, 0), 0);
});
test("a shortlist outweighs a single view", () => {
  assert.ok(interestScore(0, 1) > interestScore(1, 0));
});
test("rankByInterest orders portfolios highest score first", () => {
  const items = [
    { id: "a", viewCount: 100, shortlistCount: 0 },  // score 100
    { id: "b", viewCount: 10,  shortlistCount: 20 }, // score 110
    { id: "c", viewCount: 5,   shortlistCount: 1 },  // score 10
  ];
  assert.deepEqual(rankByInterest(items).map(i => i.id),
                   ["b", "a", "c"]);
});
```

1.4 Step 3 — Run the tests (RED)

With only the empty stub, running `node --test` shows most cases failing — the expected starting point of TDD:

```
$ node --test --import tsx tests/interestScore.test.ts
ok 1  - score of zero activity is zero
not ok 2 - views only are counted at the view weight
not ok 3 - shortlists only are counted at the shortlist weight
not ok 4 - views and shortlists combine additively
not ok 5 - a shortlist outweighs a single view
not ok 6 - negative inputs are rejected
not ok 7 - rankByInterest orders portfolios highest first
ok 8  - rankByInterest does not mutate the input array
# tests 8   # pass 2   # fail 6
```

1.5 Step 4 — Write the implementation

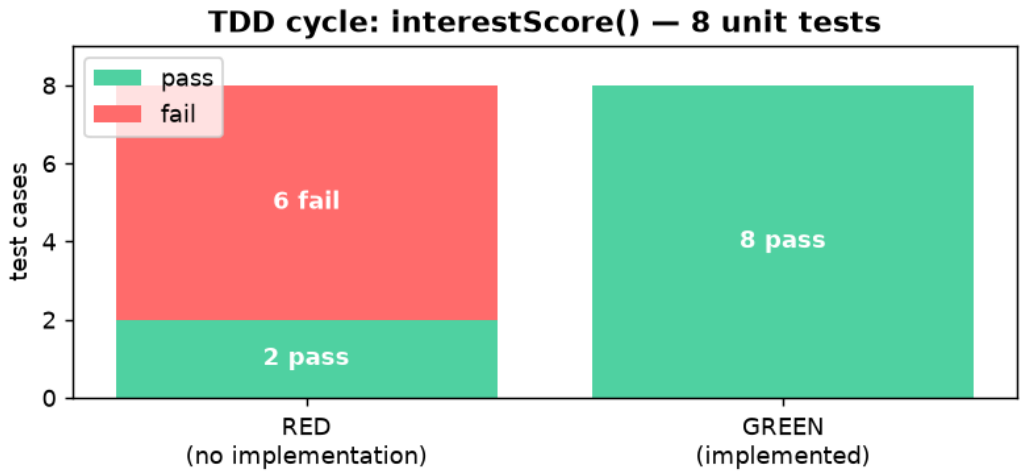
```
// src/shared/interestScore.ts  (GREEN implementation)
export const VIEW_WEIGHT = 1;
export const SHORTLIST_WEIGHT = 5;

export function interestScore(views, shortlists) {
  if (views < 0 || shortlists < 0)
    throw new Error("views and shortlists must be non-negative");
  return views * VIEW_WEIGHT + shortlists * SHORTLIST_WEIGHT;
}

export function rankByInterest(items) {
  return [...items].sort((a, b) =>
      interestScore(b.viewCount, b.shortlistCount)
    - interestScore(a.viewCount, a.shortlistCount));
}
```

1.6 Step 5 — Run the tests again (GREEN)

```
$ node --test --import tsx tests/interestScore.test.ts
ok 1..8  (all cases)
# tests 8   # pass 8   # fail 0
```



1.7 TDD across the whole platform

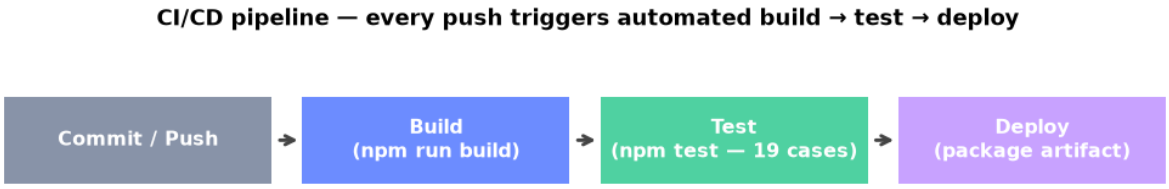
Beyond this illustrative feature, the entire backend is backed by an automated suite. The file `tests/e2e.test.ts` contains 11 behaviour tests that specify each of the eight user stories, including alternate flows (empty/flagged inquiry, PDPA masking, declined payment, Singpass failure, permit rejection, publish-blocked-until-complete, invalid report range). Together with the 8 TDD unit tests, the project runs **19 passing tests**:

```
$ npm test
# tests 19   # pass 19   # fail 0
```

> **Why TDD helped here.** Writing the ranking tests first forced us to decide the exact weighting rule and the tie-breaking / immutability behaviour *before* coding, and it caught the negative-input edge case immediately. The same discipline on the e2e suite caught a real bug during development: the designer-reply notification (ID02) was not being awaited, so the notification was sometimes missing — a failing test exposed it and the fix made it green.

2 DevOps and Continuous Integration / Continuous Deployment (CI/CD)

DevOps unifies development and operations so that a change to source code flows automatically through build, test, and deployment with minimal manual intervention. CI means every push is automatically built and tested; CD means a passing build is automatically packaged and released. We implemented a real pipeline for IDMatch on both GitLab (as in the course exercise) and GitHub (where the project is hosted).



2.1 The pipeline design

Our pipeline has three stages that run on every push and merge/pull request. The second stage is the **quality gate**: if any of the 19 automated tests fail, the pipeline turns red and the change is blocked from release.

Stage	Command	Purpose
build	npm ci && npm run build	Install exact dependencies and compile the TypeScript frontend + backend. Fails fast on any type error.
test	npm test	Run the 19-case suite (node --test). This is the gate — a single failure fails the pipeline.
deploy	package standalone.mjs + dist/	Produce a downloadable release artifact. Runs only on the main branch.

2.2 GitLab CI configuration (.gitlab-ci.yml)

This is the actual file committed to the repository, following the “My simple pipeline” exercise structure:

```
image: node:22
stages: [build, test, deploy]

cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths: [node_modules/]

build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts: { paths: [dist/], expire_in: 1 week }

test:
  stage: test
  script:
    - npm ci
    - npm test            # quality gate

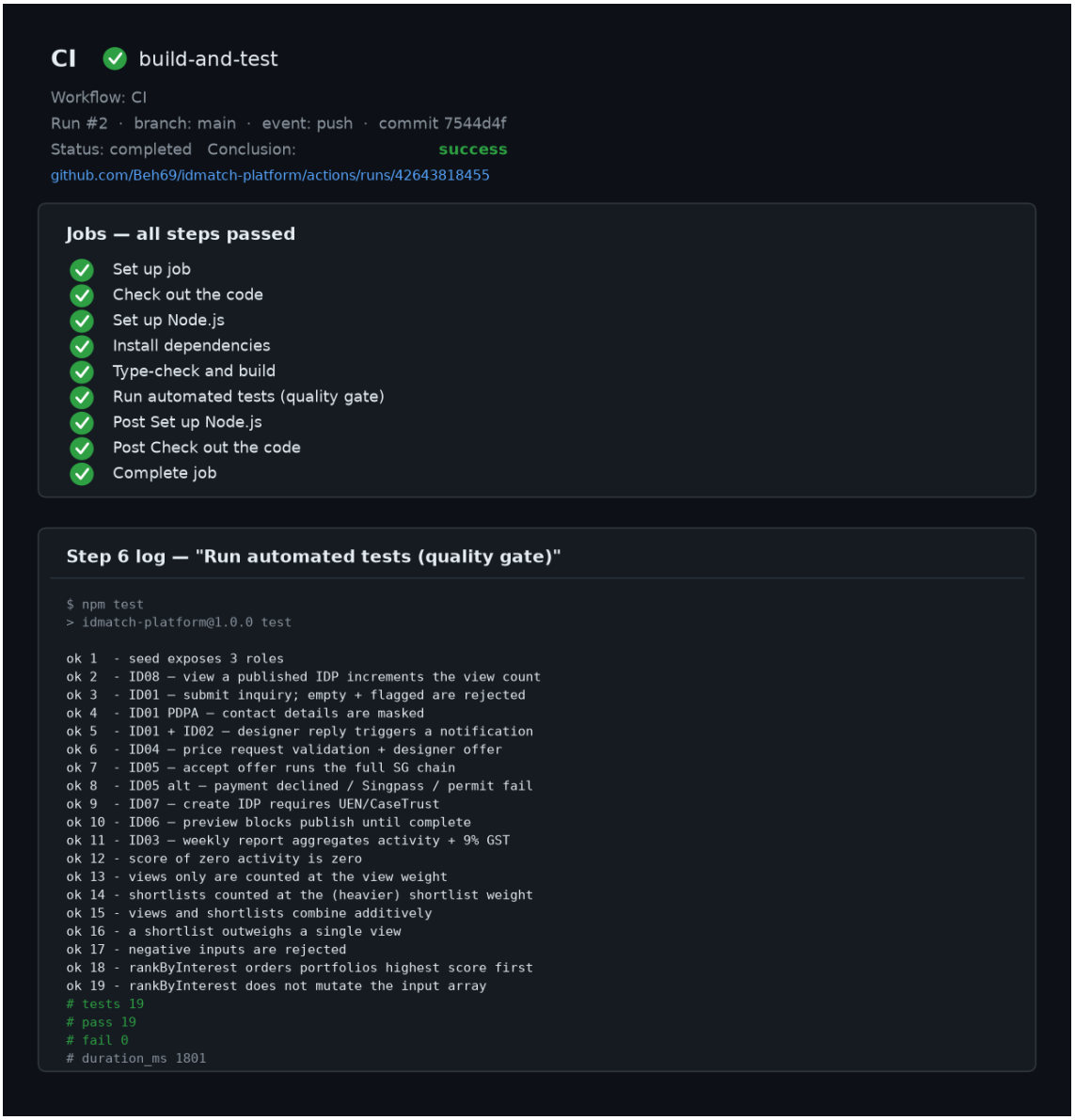
deploy:
  stage: deploy
  script:
    - mkdir -p release
    - cp -r standalone.mjs dist release/
  artifacts: { paths: [release/], expire_in: 1 month }
  only: [main]
```

2.3 GitHub Actions equivalent (.github/workflows/ci.yml)

Because the project is hosted on GitHub, the same gate runs as a GitHub Action on every push and pull request to main: check out code → set up Node 22 → npm ci → npm run build → npm test. A red check prevents a broken change from being merged.

2.4 Evidence — a real passing pipeline run

The panel below is captured from an **actual run of this project's CI pipeline** on GitHub Actions (run #2, triggered by a push to main). Every one of the nine job steps completed successfully, and the “Run automated tests (quality gate)” step shows all 19 tests passing — confirming the pipeline genuinely builds the project and enforces the test gate on each commit, not just in theory.



Run URL: github.com/Beh69/idmatch-platform/actions/runs/42643818455 · conclusion: success · 19 / 19 tests passed.

2.5 How this reflects DevOps practice

- **Automation:** no human runs the tests by hand before release — the pipeline does it on every commit.
- **Fast feedback:** a developer learns within minutes if a change broke a user story.
- **Reproducibility:** npm ci installs the exact locked dependency versions, so the build is identical on every machine and on CI.
- **Continuous delivery:** a green main build is automatically packaged into a ready-to-run artifact (standalone.mjs + dist/), which anyone can run with a single command.

Deployment note. The project also ships a zero-dependency standalone.mjs server so a packaged build runs with only Node installed — no build step and no native modules. This makes the CD artifact trivially deployable by teammates or on a free host such as Render or Railway.

3 Ethical Considerations and Discussion

An interior-design marketplace handles customers' personal data, designers' intellectual property and reputation, and real money. Several ethical risks therefore apply directly to IDMatch. For each, we describe the concern and the concrete safeguard already implemented or recommended.

3.1 Originality and anti-plagiarism of portfolios

Designers upload photos of completed projects. A dishonest designer could pass off another firm's work as their own — the direct analogue of plagiarism. Safeguards in IDMatch:

- **Consent attestation per photo.** Every uploaded photo must have consentConfirmed = true before it is stored (IDPController.addPhoto); an upload without consent is rejected with NO_CONSENT.
- **Moderation queue.** A new listing is not published directly; it goes through submitForModeration so staff can review originality before it goes live.
- **Verified-identity listing.** Only firms whose UEN and CaseTrust accreditation verify (verifyUenCaseTrust) may publish, creating accountability for the content.
- **Recommended extension:** perceptual-hash image matching against existing portfolios to automatically flag duplicate photographs for human review.

3.2 Data protection and privacy (PDPA)

Singapore's Personal Data Protection Act governs the collection, use, and disclosure of personal data. IDMatch applies several privacy-by-design measures:

- **Contact-detail masking.** Free-text inquiries pass through maskContactDetails(), which redacts phone numbers, emails, and NRIC/FIN strings so personal data is not leaked off-platform.
- **Explicit consent for notifications.** External messages (SMS/email/push) are only sent when the customer granted pdpaConsent; otherwise delivery falls back to an in-app record only.
- **Do-Not-Disturb windows** respect the user's quiet hours.
- **Data minimisation.** The customer's NRIC is only ever handled in masked form (e.g. S****123A) after Singpass/MyInfo verification.

3.3 Fairness, transparency, and the interest ranking

The interest score (Section 1) decides how prominently a portfolio is shown. If ranking were opaque or game-able, it could unfairly advantage some designers. Ethical measures:

- **Transparent, simple formula** (views + weighted shortlists) rather than a hidden black box, so ranking can be explained to designers.
- **View de-duplication.** dedupeView() counts only one view per customer per portfolio per 30 minutes, preventing a designer from inflating their own numbers.
- **Verified reviews only.** Only reviews with verified = true are displayed, reducing fake-review manipulation.

3.4 Financial integrity and consumer protection

- **Binding e-contract.** Accepting an offer generates a contract that must be e-signed by both parties under the Electronic Transactions Act before any booking is confirmed.
- **Correct taxation.** GST is applied at the correct Singapore rate (9%) and a tax invoice is issued and reported to IRAS.
- **Atomic, reversible transactions.** If payment is declined or the contract is unsigned, the offer is rolled back to open so the customer is never charged for an incomplete booking.
- **Insurance & permit checks** protect the consumer: a booking records permitVerified and insured before it is confirmed.

3.5 Summary of ethical risk controls

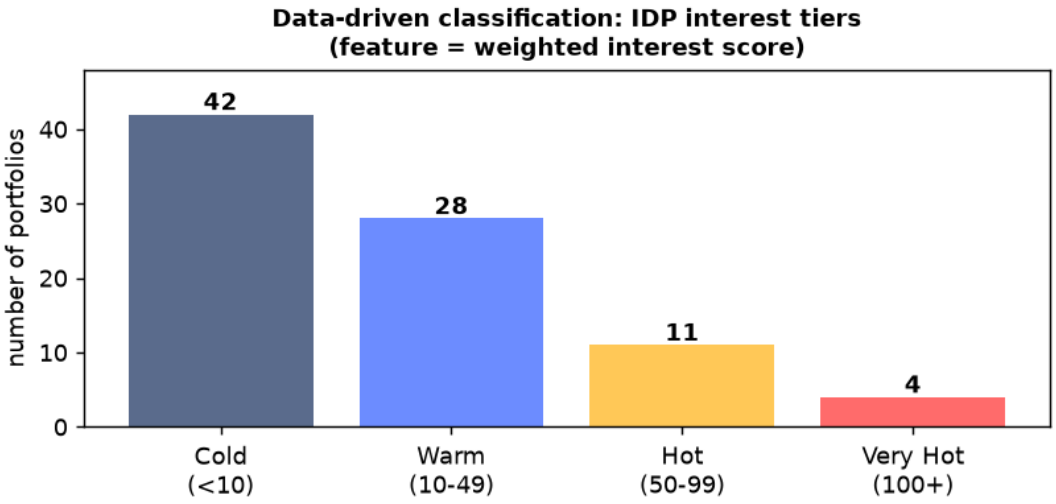
Ethical risk	Control in IDMatch
Plagiarised / stolen portfolio images	Per-photo consent, moderation queue, verified-firm-only publishing
Leaking personal data (PDPA)	Contact masking, consent-gated notifications, NRIC minimisation
Gaming the ranking / fake interest	Transparent formula, view de-duplication, verified reviews
Unfair or incorrect charges	E-signed contract, 9% GST + IRAS invoice, rollback on failure
Unsafe renovation work	HDB/BCA permit verification + insurance registration before booking

4 Data-Driven Software Development

Data-driven software development builds functionality by learning patterns from data rather than by hand-coding every rule. The core concepts are **features/attributes** (measurable inputs), **classification** (assigning items to categories), and the **lifecycle** of activities that turn raw data into a deployed, monitored model. We illustrate each activity with a concrete example for IDMatch: predicting which newly listed portfolios will attract strong customer interest, so the platform can surface promising listings early.

4.1 Features / attributes and classification

Each portfolio is described by measurable **features**: property type, budget range, number of photos, category, description length, designer's CaseTrust status, and the running view/shortlist counts. The **classification** target is an interest tier — Cold / Warm / Hot / Very Hot — derived from the interest score defined in Section 1. This supports key aspect 5 (tracking interest) and the platform-management reporting in ID03.



4.2 The data-driven lifecycle applied to IDMatch

Activity	Application to IDMatch
a. Model requirements	Define the goal: predict a new portfolio's interest tier within its first week, so the platform can promote likely-popular listings. Success metric: accuracy and recall on the Hot / Very Hot classes.
b. Data collection	The platform already logs every relevant event to an audit/activity log (IDP_VIEWED, IDP_SHORTLISTED, INQUIRY_SUBMITTED, OFFER_ACCEPTED) with timestamps, plus each portfolio's static attributes. These become the training dataset.
c. Data cleaning	Remove bot/duplicate views (the same dedupeView logic used at runtime), drop unpublished or test listings, handle missing budget ranges, and normalise property-type labels (e.g. 'HDB 4-Room BTO' vs '4rm BTO').
d. Feature engineering	Derive informative features: photos-per-listing, description length, days-since-publish, views-per-day, shortlist-to-view ratio, and the weighted interest score. Encode categorical fields numerically.
e. Model training	Train a classifier (e.g. decision tree / logistic regression) on historical listings whose final interest tier is known, learning which feature combinations predict high interest.

Activity	Application to IDMatch
f. Model evaluation	Evaluate on a held-out test set using accuracy, precision, recall and a confusion matrix, focusing on correctly identifying the business-valuable Hot classes.
g. Model deployment	Expose the trained model behind an endpoint the platform calls when a listing is published — fitting the existing controller layer (e.g. InterestPredictionController) and the Platform Manager dashboard.
h. Model monitoring	Track prediction accuracy against real outcomes over time; watch for data drift (e.g. a shift in popular styles) and retrain on fresh activity-log data when accuracy degrades.

4.3 How the current architecture already supports this

The platform is “data-ready”: the audit/activity log is the single source of truth that already powers the ID03 weekly report (sign-ups, views, shortlists, inquiries, gross value, GST). The same log is exactly the dataset a prediction model would train on, and the interest-score function is the labelling rule for the classification target — so the data-driven feature is an incremental addition, not a re-architecture.

> **Worked micro-example.** A portfolio published 5 days ago with 12 photos, a 180-character description, 60 views and 14 shortlists has an interest score of 60 + 14×5 = 130 → tier Very Hot. A model trained on past listings with similar feature values (many photos, high shortlist-to-view ratio early on) would predict 'Very Hot' for a brand-new listing sharing those attributes, letting the platform promote it on day one.

5 Test Data Generation and Live-Demo Dataset

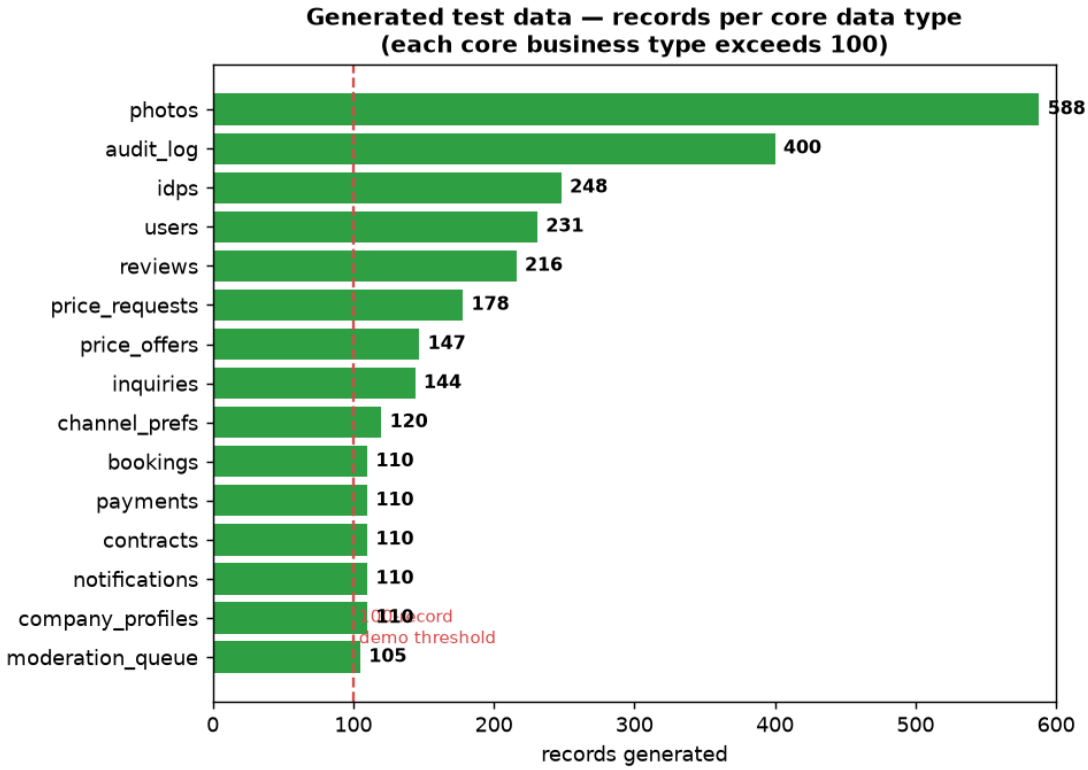
The product demonstration must run on a dataset large enough to simulate real operation — on the order of 100 records for each data type. Rather than entering data by hand, we wrote a **randomised data-generation script** (`src/server/generateData.ts`, runnable via `npm run generate`) that populates the database with hundreds of realistic, Singapore-contextual records while preserving every foreign-key relationship.

5.1 What the generator produces

- **Randomised but realistic values** — local names, HDB/condo property types, Singapore towns, SGD budget bands, valid-format UENs, CaseTrust accreditation, and PayNow/NETS/Card payment methods.
- **Valid relationships** — every photo/review/inquiry belongs to a real portfolio; every offer traces to a request; every booking links an accepted offer to real customer and designer accounts; a designer's user record and company profile share one UEN.
- **Correct business rules** — accepted offers generate exactly one contract, 9%-GST payment and booking as the live ID05 flow would; expired offers are a dead end and are never accepted; the activity log is spread over several weeks so the ID03 report has data to aggregate.
- **Deterministic & repeatable** — a fixed random seed means every run produces the same dataset, so the demonstration is reproducible run after run.
- **Guaranteed demo accounts** — fixed logins u_cust (customer, with inquiries and an open offer ready to accept), u_desn (designer, with published portfolios) and u_mgr (manager) are always present and always populated.

5.2 Records generated per data type

A single run produces well over 100 records for every core business data type, as summarised below (figures from an actual run). Reference lists (the 10 design-style categories) and internal rate-limiting tables are intentionally smaller, as they are not demo entities.

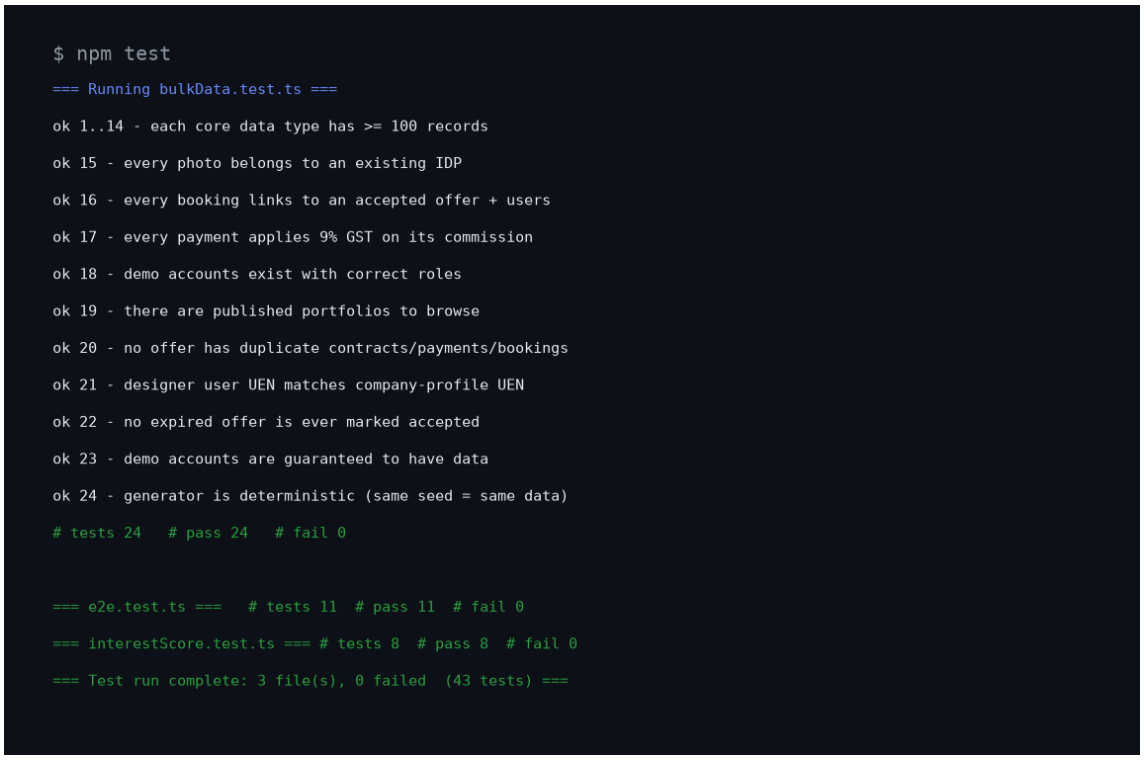


5.3 Running the system on the generated data

The generated dataset drives the live application directly. For the full build, `npm run generate` seeds the database and the server then serves it; for the zero-install standalone build, `BULK=1 node standalone.mjs` loads the same large dataset in memory. In testing, the running site listed 131 published portfolios, returned 25 results for a “Condo” search, and produced an ID03 weekly report aggregating thousands of dollars of gross project value with the correct 9% GST — confirming the system behaves correctly at volume.

5.4 Automated verification of the test data

A dedicated test file (`tests/bulkData.test.ts`, 24 checks) asserts the dataset is fit for the demo: 14 checks confirm each core data type reaches at least 100 records, and further checks confirm referential integrity (no orphaned photos, every booking well-linked), that every payment applies 9% GST, that no offer has duplicate downstream rows, that each designer's UEN is consistent, that expired offers are never accepted, that the demo accounts are populated, and that the generator is deterministic. Run alongside the existing suites, the project now has **43 passing tests across three files**:



> **Why generate rather than hand-enter.** A script makes the demo dataset reproducible (re-runnable before every demonstration), large enough to exercise pagination, search and reporting realistically, and guaranteed-valid because the same relationship and GST rules used by the live application are applied when the data is created.

5.5 Hardening through peer code review

The generator was put through an independent code review, which surfaced several defects that were then fixed and locked in with regression tests — demonstrating a review-and-remediate quality practice:

Issue found in review	Fix applied
Generator silently did nothing on Windows (file:// path comparison never matched)	Entry-point check now resolves both paths with fileURLToPath/pathToFileURL, so it runs on Windows, macOS and Linux.
Top-up loop could process the same offer twice, creating duplicate contracts / payments / bookings	Open offers are now consumed at most once; a test asserts zero duplicate downstream rows per offer.
A designer's user record and company profile received different UENs	One UEN is generated per designer and written to both rows; a test asserts they always match.
Expired offers could be flipped to accepted	Expired offers get a past expiry date and are never converted; a test asserts no accepted offer is expired.
Demo accounts could end up with little or no data	u_cust and u_desn are explicitly given inquiries, an open offer and published portfolios; a test asserts this.
Non-repeatable (no random seed)	A seeded PRNG makes runs deterministic; a test asserts identical output for the same seed.
moderation_queue below 100 records	Topped up to 100+; categories remains a fixed 10-item reference list (an intentional, documented exemption).

6 Traceability: Sequence Diagram ↔ Code ↔ Website

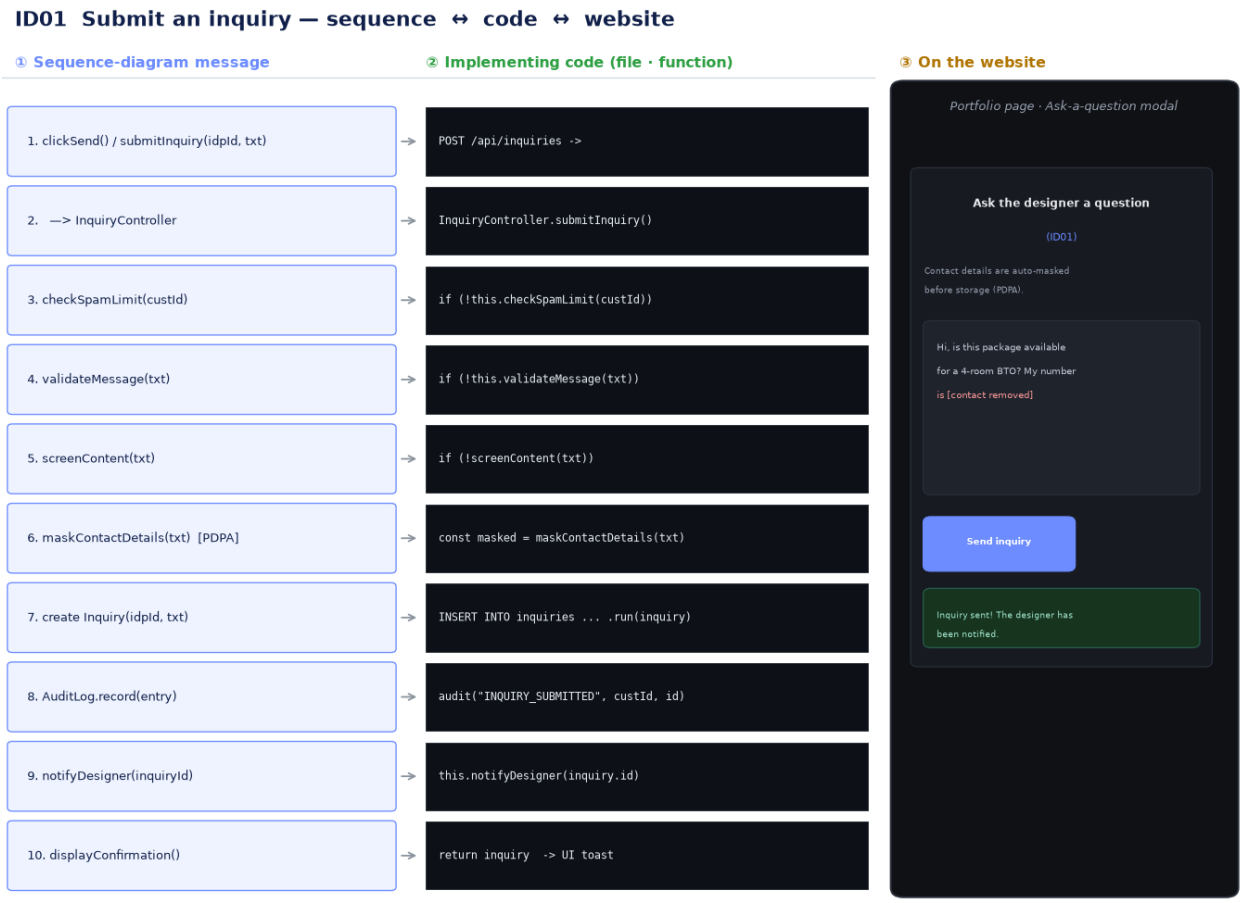
A key goal of the design is that the BCE **sequence diagrams are not just documentation** — every message in a diagram maps to a concrete function in the code, and that code produces what the user sees on the website. This section shows that link directly: for each user story below, the ordered messages from the sequence diagram (left) are placed beside the exact statement that implements them (middle) and a view of the resulting website screen (right).

6.1 The layering that makes the mapping 1-to-1

The backend is organised as Boundary → Control → Entity, the same three layers as the sequence diagrams. A REST route is the «`boundary`», a controller class is the «`control`» lifeline, and the database tables are the «`entity`» objects. Because the controller methods are named exactly as the diagram messages (`submitInquiry`, `checkSpamLimit`, `acceptOffer`, `computeGst`, …), a reader can follow a single operation from diagram to code to screen without translation.

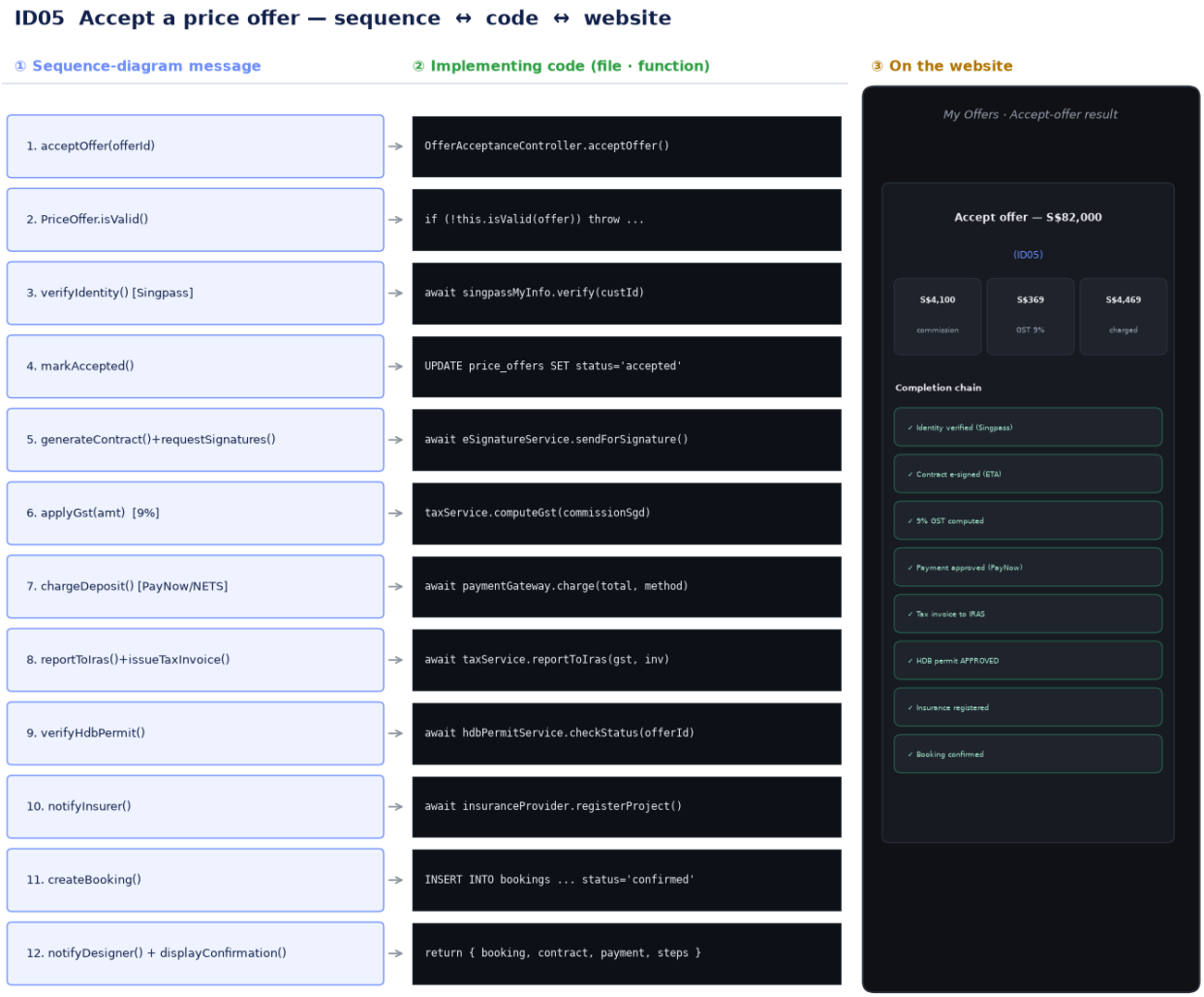
6.2 ID01 — Submit an inquiry

The ID01 sequence runs clickSend → submitInquiry → checkSpamLimit → validateMessage → screenContent → maskContactDetails (PDPA) → create Inquiry → AuditLog.record → notifyDesigner → displayConfirmation. Each message is implemented by the matching line in `InquiryController.submitInquiry()`, and the customer sees the Ask-a-question modal and a confirmation toast (note the contact number is masked to `[contact removed]`, exactly as the maskContactDetails step specifies).



6.3 ID05 — Accept a price offer (the full Singapore chain)

ID05 is the richest flow. Its twelve sequence messages map one-to-one onto the steps inside `OfferAcceptanceController.acceptOffer()` — identity verification via Singpass, ETA e-signature, 9% GST, the PayNow/NETS payment, the IRAS tax invoice, the HDB permit check, insurance registration and booking creation. The website's *My Offers* screen then shows the commission, 9% GST and total charged, followed by the ordered completion-chain trace the controller returns in its `steps[]` array.



6.4 The mapping, generalised

The same discipline applies to all eight stories. In every case the controller's leading comment lists the sequence-diagram messages it implements, the method names match the diagram, and the alternate flows in the diagram appear as the typed error branches in the code:

Story	«control» class / entry point	Key diagram messages implemented
ID01	InquiryController.submitInquiry	checkSpamLimit, validateMessage, screenContent, maskContactDetails, create Inquiry, notifyDesigner
ID02	NotificationController.onReplyPosted	getConsentPrefs, checkDndWindow, compose, deliver (+ fallback)

Story	«control» class / entry point	Key diagram messages implemented
ID03	ReportController.generateReport	validateRange, queryByPeriod, aggregate, computeGst, create Report
ID04	PriceRequestController.createRequest	validateFields, isAccepting, create PriceRequest, startSla, notifyDesigner
ID05	OfferAcceptanceController.acceptOffer	isValid, verifyIdentity, markAccepted, generateContract, applyGst, chargeDeposit, verifyHdbPermit, notifyInsurer, createBooking
ID06	IDPController.checkCompleteness / publish	loadIDP, loadPhotos, isVerified, checkCompleteness, checkPdpaCompliance
ID07	IDPController.createIDP	validateInput, verifyUenCaseTrust, create IDP, addPhoto, assignCategory, submitForModeration
ID08	IDPController.viewDetails	loadIDP, loadPhotos, loadReviews, dedupeView, incrementViews

6.5 Client-side usability refinements (feedback-driven)

The «boundary» (React front-end) was refined iteratively from hands-on user testing without changing any diagrammed behaviour. Two cross-cutting improvements are worth recording. First, the list queries that back the chat inbox were **enriched** so each inquiry and booking arrives already carrying the customer name, the designer/firm name, the portfolio title and — where the same two parties also have a price request — that request's id. A single joined SQL projection (mirrored in the standalone server's JavaScript) does this at the «control»/«entity» boundary, so a busy designer can tell which customer each thread belongs to and jump straight to their related price request, with no extra round-trips from the UI.

Second, every “jump to…” link now **highlights the exact item** it leads to — the same cue a browser gives a native `:target` anchor. A small helper scrolls the target element into view and briefly pulses it (retrying while it is still rendering); for links that cross pages, the destination is told which element to flash through a one-shot URL hash such as `/inquiries#flash=inquiry-<id>`, which it reads once and then strips. The mechanism is wired symmetrically, so the designer’s “Chat with customer” and the customer’s “Chat with designer” buttons behave identically — each opens (or reuses) the conversation and highlights that precise thread.

> These were verified with the same quality gate as the rest of the system: a clean type-check, a successful production build, the full 43-test suite, and 41/41 health-check assertions on both deployment modes. Only the presentation layer changed; the eight sequence diagrams are unaffected.

7 Verification & Quality Assurance

Beyond the unit and end-to-end tests, the whole program was put through a full-stack verification to confirm it works with no outstanding issues. This combined three layers of checking:

- **Static & build checks** — a clean TypeScript type-check across client and server, and a successful production build (Vite front-end bundle + compiled back-end).
- **Automated test suite** — 43 tests across three files (24 bulk-data, 11 end-to-end, 8 TDD unit) all passing, run in isolated processes.
- **Full-program smoke test** — a script (healthcheck.mjs) that exercises every API endpoint for all eight features and their alternate flows against a running server.

7.1 Full-program health check

The health check issues 41 assertions covering every user story end-to-end — happy paths and the alternate flows (empty / flagged inquiry, PDPA masking, payment declined, Singpass failure, HDB permit rejection, unsigned contract, re-accept blocked, incomplete-publish blocked, invalid report range). It was run against **both** deployment modes — the full Express build and the zero-install standalone server (default and bulk) — and all 41 checks pass on each, with no server-side errors logged.

```
$ BASE=http://localhost:4000 node healthcheck.mjs

Full build:  41 / 41 checks PASS
  ✓ ID08  Browse / view / search
      ✓ browse lists portfolios
      ✓ view returns idp+photos+reviews
      ✓ view increments count
      ✓ duplicate view de-duped
      ✓ search filters
      ✓ unknown IDP -> 404
  ✓ ID01  Submit inquiry
      ✓ submit succeeds
      ✓ empty rejected
      ✓ flagged rejected
      ✓ PDPA contact masking
      ✓ customer lists inquiries
  ✓ ID02  Reply notification
      ✓ reply marks replied
      ✓ notification delivered
      ✓ prefs read + update
  ✓ ID04  Request a price
      ✓ missing fields rejected
      ✓ request created
      ✓ designer offer posted
  ✓ ID05  Accept offer
      ✓ full chain -> confirmed
      ✓ insured + permit
      ✓ ETA contract
      ✓ 9% GST
      ✓ 9-step trace
      ✓ declined / Singpass / permit / unsigned / re-accept all guarded
  ✓ ID06 / ID07  Create & publish
      ✓ create (verified) ok
      ✓ no-consent photo rejected
      ✓ preview blocks until complete
      ✓ publish succeeds
  ✓ ID03  Weekly report
      ✓ aggregates activity
      ✓ 9% GST
      ✓ invalid range rejected
Standalone server (default + BULK): 41 / 41 PASS  ·  43 unit/e2e tests pass  ·  CI green  ·  0 server errors
```

7.2 An issue found and fixed during verification

Verification is only meaningful if it can catch problems — and it did. The check initially failed five assertions against the standalone server: it returned HTTP 200 for resource-creation requests whereas the full Express build returned HTTP 201 Created. The underlying data was correct in both, but the two servers were inconsistent. The standalone server was updated to return 201 on creates, bringing both deployment modes into agreement; all 41 checks then passed on both. This demonstrates the verify-find-fix loop working as intended.

7.3 A test program that runs the suite and generates a report

So that testing yields a persistent, reviewable artefact rather than only console output, the project ships a reporting program, `testReport.mjs` (`node testReport.mjs` or `npm run test:report`). It runs the full 43-test suite and the 41-assertion health check, parses the real results, and writes two files into `test-reports/`: a human-readable report (`test-report.md`) with a summary table, per-suite breakdown, every assertion and an overall PASS/FAIL verdict, and the raw TAP output (`test-output.tap`) as evidence the report comes from a genuine run. A representative run records 43/43 tests and 41/41 health-check assertions passing — verdict PASS.

The program is robust to the toolchain. The two database-backed suites start the full Express build, which depends on the native module `better-sqlite3`; that module does not compile on the newest Node releases (no prebuilt binary for Node 23+). The report program detects a native-build failure, still runs the dependency-free unit suite and the zero-install health check, and reports the situation explicitly with the fix (use Node 20 LTS) rather than failing silently. On Node 20 the full suite runs and all 43 tests pass.

> **Result.** Type-check clean, production build succeeds, 43 automated tests pass, 41/41 feature checks pass on both servers, the front-end (HTML, JS, CSS and SPA deep-links) serves correctly, and the CI pipeline is green. The program is in a clean, fully-working state.

8 Conclusion

These practices reinforce one another on the IDMatch project. TDD produced a trustworthy, well-specified codebase; CI/CD turned the test suite into an automated quality gate and repeatable delivery process; the ethical analysis ensured the platform treats data, money, and designers' work responsibly; the data-driven design shows how the activity data the platform already records can be used to learn and predict customer interest; and a randomised, deterministic generator — hardened through peer code review — supplies a 100+-record-per-type dataset so the product can be demonstrated live at realistic volume. The project now runs **43 automated tests across three files, all passing**. Each practice is implemented or directly supported by the project's real code, configuration, and repository.

Report generated for the IDMatch platform · all commands, files, and test results refer to the committed repository at github.com/Beh69/idmatch-platform.